

Praktikum 2: Laporan Praktikum 2 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:  Praktikum02.ipynb

Link GitHub:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum02/>

1. Sinkronisasi dengan Drive

Langkah pertama adalah menghubungkan *environment* Google Colab dengan Google Drive. Tujuannya adalah agar *notebook* Colab dapat mengakses file dataset yang tersimpan di dalam akun Drive

```
# menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/gdrive')
```

 Mounted at /content/gdrive

Proses ini berhasil dengan output **Mounted at /content/gdrive**, menandakan direktori Drive sudah siap diakses.

2. Membaca file csv dengan pandas

Setelah Drive terhubung, langkah selanjutnya adalah membaca file **500_Person_Gender_Height_Weight_Index.csv** menggunakan *library* Pandas.

```
# membaca file csv menggunakan pandas
import pandas as pd

df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum02/datasets/500_Person_Gender_Height_Weight_Index.csv')
df
```



	Gender	Height	Weight	Index
0	Male	174	96	4
1	Male	189	87	2
2	Female	185	110	4
3	Female	195	104	3
4	Male	149	61	3
...	...	...	...	...
495	Female	150	153	5
496	Female	184	121	4
497	Female	141	136	5
498	Male	150	95	5
499	Male	173	131	5

500 rows x 4 columns

- Fungsi `pd.read_csv()` digunakan untuk memuat data dari file CSV ke dalam sebuah struktur data yang disebut DataFrame.

- DataFrame `df` ini pada dasarnya adalah sebuah tabel yang berisi seluruh data dari file CSV, yang terdiri dari 500 baris dan 4 kolom. Kolom-kolom tersebut adalah 'Gender', 'Height', 'Weight', dan 'Index'.

3. Melihat Info Data

```
# mencari info data pada file (tipe datanya, non null count data, nama kolom)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Gender  500 non-null       object
1   Height  500 non-null       int64
2   Weight  500 non-null       int64
3   Index   500 non-null       int64
dtypes: int64(3), object(1)
memory usage: 15.8+ KB
```

Fungsi ini memberikan ringkasan teknis dari DataFrame:

- Terdapat 500 entri data, dari indeks 0 hingga 499.
- Terdapat 4 kolom: 'Gender', 'Height', 'Weight', dan 'Index'.
- Semua kolom memiliki 500 data non-null, artinya tidak ada data yang kosong atau hilang.
- Kolom 'Gender' bertipe `object` (teks), sementara sisanya bertipe `int64` (angka bulat).

4. Mencari nilai Mean (nilai rata-rata)

Analisis statistik dimulai dengan mencari nilai rata-rata (mean) dari kolom 'Height'. Mean berguna untuk mengetahui nilai pusat dari data.

```
[ ] # menghitung mean semua kolom numerik
    df['Height'].mean()
```

```
np.float64(169.944)
```

Diperoleh nilai mean untuk tinggi badan sebesar **169.944**. Ini berarti, rata-rata tinggi badan dari 500 orang dalam dataset ini adalah sekitar 170 cm.

5. Mencari nilai Median (nilai tengah)

Median dicari untuk menemukan nilai tengah dari data tinggi badan setelah diurutkan. Median lebih kebal terhadap nilai ekstrem (outlier) jika dibandingkan dengan mean.

```
[ ] # menghitung median semua kolom numerik
    df['Height'].median()
```

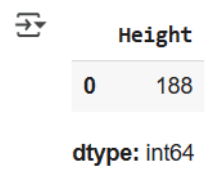
```
170.5
```

Nilai median dari kolom 'Height' adalah **170.5**. Artinya, 50% dari data tinggi badan berada di bawah 170.5 cm, dan 50% sisanya berada di atasnya.

6. Mencari nilai Modus (nilai yang paling sering muncul)

Modus digunakan untuk mengetahui nilai manakah yang memiliki frekuensi kemunculan tertinggi dalam data tinggi badan.

```
[ ] # menghitung modus
df['Height'].mode()
```



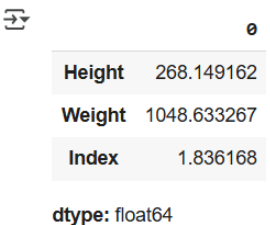
The output shows a single value, 188, for the 'Height' column. The data type is int64.

Nilai modus yang didapatkan adalah **188**. Ini menunjukkan bahwa tinggi badan 188 cm adalah yang paling banyak ditemukan dalam dataset ini.

7. Menghitung Variansi

Variansi dihitung untuk mengukur seberapa tersebar data dari nilai rata-ratanya. Semakin besar nilainya, semakin bervariasi datanya

```
[ ] # menghitung variansi
df.var(numeric_only=True)
```



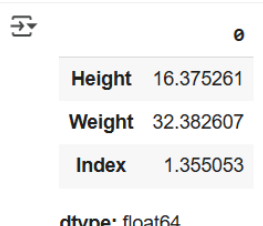
The output shows the variance for three columns: Height (268.149162), Weight (1048.633267), and Index (1.836168). The data type is float64.

Diperoleh nilai variansi untuk setiap kolom numerik: 'Height' (268.149), 'Weight' (1048.633), dan 'Index' (1.836).

8. Menghitung Standar Deviasi

Standar deviasi (simpangan baku) juga merupakan ukuran sebaran data, namun dalam satuan yang sama dengan data aslinya (akar kuadrat dari variansi), sehingga lebih mudah diinterpretasikan.

```
[ ] # menghitung standar deviasi
df.std(numeric_only=True)
```



The output shows the standard deviation for three columns: Height (16.375261), Weight (32.382607), and Index (1.355053). The data type is float64.

Nilai standar deviasi untuk 'Height' adalah sekitar **16.37**, 'Weight' **32.38**, dan 'Index' **1.35**.

9. Menghitung Q1, Q3, dan IQR

Langkah ini bertujuan untuk membagi data menjadi empat bagian sama besar untuk memahami distribusinya lebih dalam.

- **Q1 (Kuartil 1):** Nilai yang membatasi 25% data terendah.
- **Q3 (Kuartil 3):** Nilai yang membatasi 75% data terendah.
- **IQR (Interquartile Range):** Jangkauan antara Q3 dan Q1, yang menunjukkan sebaran 50% data di bagian tengah.

```
[ ]  
  
# hitung kuartil pertama (Q1)  
q1 = df['Height'].quantile(0.25)  
print("Q1: ", q1)  
  
# hitung kuartil ketiga (Q3)  
q3 = df['Height'].quantile(0.75)  
print("Q3: ", q3)  
  
# hitung IQR (Interquartile Range)  
iqr = q3 - q1  
print("IQR: ", iqr)
```

```
Q1: 156.0  
Q3: 184.0  
IQR: 28.0
```

Diperoleh Q1 = **156.0** , Q3 = **184.0** , dan IQR = **28.0**.

10. Membuat Deskripsi Statistika

Fungsi `describe()` digunakan untuk menampilkan rangkuman lengkap statistik deskriptif dari semua kolom numerik secara otomatis.

```
[ ]  
  
# untuk membuat statistika deskripsi pada type data int  
df.describe()
```

```
Height      Weight      Index  
count  500.000000  500.000000  500.000000  
mean    169.944000  106.000000   3.748000  
std      16.375261   32.382607   1.355053  
min     140.000000   50.000000   0.000000  
25%     156.000000   80.000000   3.000000  
50%     170.500000  106.000000   4.000000  
75%     184.000000  136.000000   5.000000  
max     199.000000  160.000000   5.000000
```

Outputnya berupa tabel yang merangkum jumlah data (**count**), rata-rata (**mean**), standar deviasi (**std**), nilai minimum (**min**), kuartil (25%, 50%, 75%), dan nilai maksimum (**max**) untuk kolom 'Height', 'Weight', dan 'Index'.

11. Menghitung dan menampilkan Matriks Korelasi

Matriks korelasi dibuat untuk mengukur kekuatan dan arah hubungan linear antar variabel numerik. Nilai korelasi berkisar dari -1 (berbanding terbalik) hingga +1 (berbanding lurus).

```
[ ]  
  
# menghitung matriks korelasi semua kolom numerik  
correlation_matrix = df.corr(numeric_only=True)  
  
# menampilkan matriks korelasi  
print("Matriks Korelasi:")  
print(correlation_matrix)
```

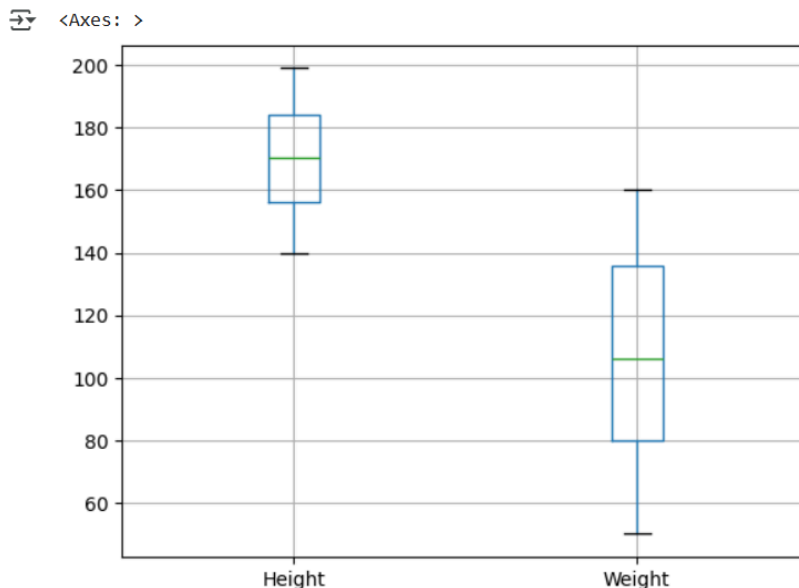
```
↗ Matriks Korelasi:  
      Height  Weight  Index  
Height 1.000000 0.000446 -0.422223  
Weight 0.000446 1.000000 0.804569  
Index -0.422223 0.804569 1.000000
```

Dari matriks korelasi, terlihat bahwa ada hubungan positif yang kuat antara 'Weight' dan 'Index' dengan nilai **0.804569**. Sebaliknya, hubungan antara 'Height' dan 'Index' bersifat negatif sedang (-0.422223).

12. Membuat visualisasi data dengan Boxplot

Boxplot digunakan untuk memvisualisasikan distribusi data 'Height' dan 'Weight' secara grafis. Grafik ini efektif untuk melihat sebaran data, median, kuartil, dan potensi adanya outlier.

```
[ ]  
import pandas as pd  
import numpy as np  
  
df.boxplot(column=['Height', 'Weight'])
```



Visualisasi boxplot menunjukkan ringkasan statistik dari kedua kolom. Dari grafik tersebut, kita bisa membandingkan rentang nilai dan sebaran data tinggi badan dan berat badan secara visual.

13. Membuat visualisasi data dengan Histogram

Histogram dibuat khusus untuk kolom 'Height' untuk melihat distribusi frekuensinya. Grafik ini menunjukkan seberapa sering rentang nilai tinggi badan tertentu muncul dalam data.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# ambil data height
data_height = df["Height"]

# buat histogram
n, bins, patches = plt.hist(data_height, bins=5, color='pink', edgecolor='black')

# tambahkan label
plt.title('Histogram Nilai')
plt.xlabel('Height')
plt.ylabel('Frekuensi')

# tampilkan rentang frekuensi di sumbu x
bin_centers = 0.5*(bins[:-1] + bins[1:])
plt.xticks(bin_centers, ['{:0f}-{:0f}'.format(bins[i], bins[i+1]) for i in range(len(bins)-1)])

# tampilkan histogram
plt.show
```

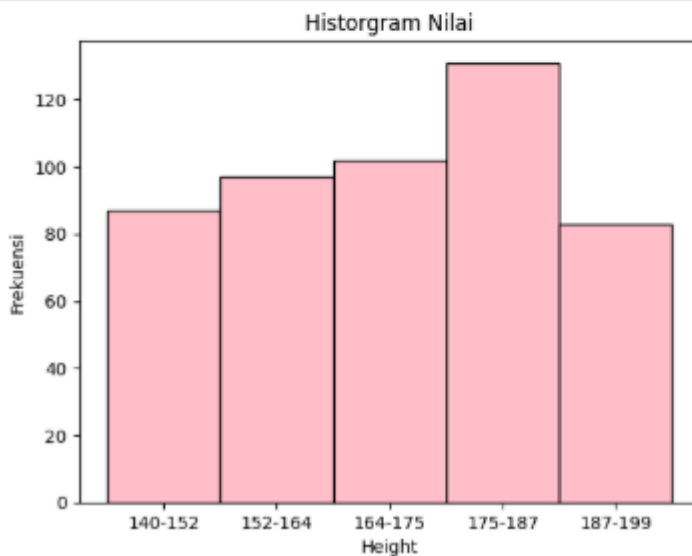


```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

Display all open figures.

Parameters

block : bool, optional
Whether to wait for all figures to be closed before returning.



Histogram yang dihasilkan menunjukkan bahwa rentang tinggi badan antara **175-187 cm** memiliki frekuensi tertinggi, yaitu lebih dari 120 data.

14. Membuat visualisasi data dengan Scatter Plot menggunakan data Nilai1 dan Nilai2 untuk Korelasi Positif

Scatter plot digunakan untuk memvisualisasikan hubungan antara dua variabel. Pada praktikum ini, dibuat dua contoh scatter plot menggunakan data buatan untuk mengilustrasikan korelasi positif dan negatif.

```

import pandas as pd
import matplotlib.pyplot as plt

# buat dataframe contoh
data = {
    'Nilai1': [1,2,3,4,5,6,7,8,9,10],
    'Nilai2': [2,4,6,8,10,12,14,16,18,20]
}

df2 = pd.DataFrame(data)

# buat scatter plot
plt.scatter(df2['Nilai1'], df2['Nilai2'], color='blue', marker='o')

# tambahkan label
plt.title('Scatter Plot Korelasi Positif')
plt.xlabel('Nilai1')
plt.ylabel('Nilai2')

# tambahkan grid
plt.grid(True)

# tampilkan plot
plt.show

```

(4)

```

matplotlib.pyplot.show
def show(*args, **kwargs) -> None

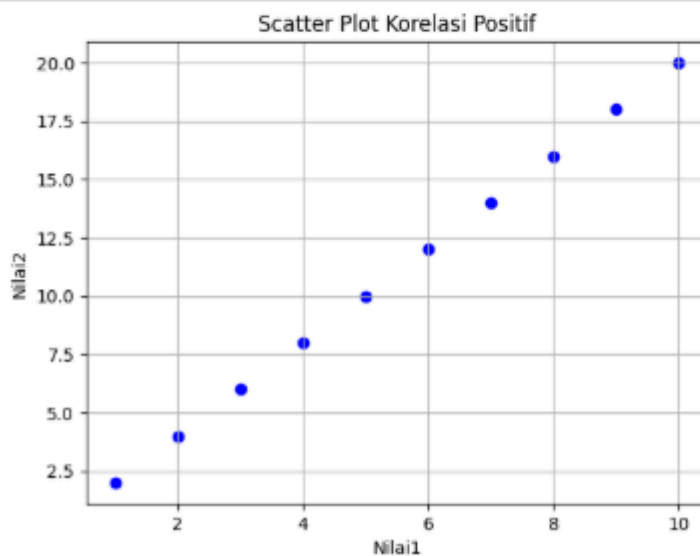
```

Display all open figures.

Parameters

block : bool, optional

Whether to wait for all figures to be closed before returning.



Scatter Plot Korelasi Positif: Grafik menunjukkan pola titik yang naik dari kiri bawah ke kanan atas. Ini mengindikasikan bahwa saat 'Nilai1' bertambah, 'Nilai2' juga ikut bertambah.

15. Membuat visualisasi data dengan Scatter Plot menggunakan Nilai1 dan Nilai2 untuk Korelasi Negatif

```
import pandas as pd
import matplotlib.pyplot as plt

# buat dataframe contoh
data = {
    'Nilai1': [1,2,3,4,5,6,7,8,9,10],
    'Nilai2': [10,9,8,7,6,5,4,3,2,1]
}

df3 = pd.DataFrame(data)

# buat scatter plot
plt.scatter(df3['Nilai1'], df3['Nilai2'], color='red', marker='x')

# tambahkan label
plt.title('Scatter Plot Korelasi Negatif')
plt.xlabel('Nilai1')
plt.ylabel('Nilai2')

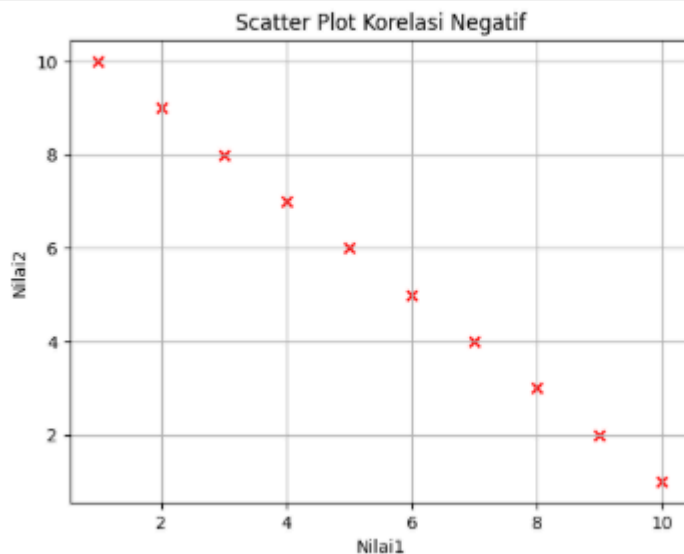
# tambahkan grid
plt.grid(True)

# tampilkan plot
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None

Display all open figures.

Parameters
-----
block : bool, optional
    Whether to wait for all figures to be closed before returning.
```



Scatter Plot Korelasi Negatif: Grafik ini menunjukkan pola titik yang menurun dari kiri atas ke kanan bawah. Ini berarti saat 'Nilai1' bertambah, 'Nilai2' justru berkurang.

Tugas Mandiri 2: Laporan Praktikum Mandiri 2 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab: [Tugas2.ipynb](#)

Link GitHub:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum02>

1. Sinkronisasi Drive, Import Library, Load Data, Inspeksi Awal

```
# Hubungkan Google Colab ke Google Drive
from google.colab import drive
drive.mount('/content/gdrive')

# Import pandas
import pandas as pd

# Baca file CSV dari drive
df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum02/datasets/day.csv')

# Tampilkan jumlah baris dan kolom
print("Jumlah data (rows, cols):", df.shape)

# Tampilkan 5 baris pertama
df.head()
```

Mounted at /content/gdrive
Jumlah data (rows, cols): (731, 16)

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
0	1	2011-01-01	1	0	1	0	6	0	2	0.344167	0.363625	0.805833	0.160446	331	654	985
1	2	2011-01-02	1	0	1	0	0	0	2	0.363478	0.353739	0.696087	0.248539	131	670	801
2	3	2011-01-03	1	0	1	0	1	1	1	0.196364	0.189405	0.437273	0.248309	120	1229	1349
3	4	2011-01-04	1	0	1	0	2	1	1	0.200000	0.212122	0.590435	0.160296	108	1454	1562
4	5	2011-01-05	1	0	1	0	3	1	1	0.226957	0.229270	0.436957	0.186900	82	1518	1600

- **Sinkronisasi Drive:** Perintah `drive.mount()` digunakan untuk menghubungkan Google Colab dengan Google Drive. Ini adalah tahap penting agar *notebook* dapat mengakses dan membaca file dataset `day.csv` yang tersimpan di dalam Drive.
- **Import Library:** *Library* Pandas diimpor dengan alias `pd`. Pandas adalah alat utama dalam analisis data di Python yang digunakan untuk memanipulasi data dalam format tabel (DataFrame).
- **Pemuatan Data:** Data dari file `day.csv` dibaca dan dimuat ke dalam sebuah DataFrame bernama `df` menggunakan fungsi `pd.read_csv()`.
- **Inspeksi Awal:**
 - Untuk mengetahui ukuran dataset, perintah `df.shape` dijalankan. Hasilnya menunjukkan bahwa dataset ini terdiri dari 731 baris (data) dan 15 kolom (fitur).
 - Fungsi `df.head()` digunakan untuk menampilkan lima baris pertama dari data. Ini berguna untuk mendapatkan gambaran awal mengenai isi dan struktur kolom-kolom yang ada, seperti `dteday`, `season`, `temp`, dan `cnt` (jumlah total sewa).

2. Membagi Data untuk Train 80% dari Total Dataset

```
# Split awal: ambil Train = 80% dan sisakan Test = 20%
from sklearn.model_selection import train_test_split

RANDOM_STATE = 42

# Train
train_df, test_df = train_test_split(df, test_size=0.2, random_state=RANDOM_STATE, shuffle=True)

print("Step 1: Buat Train (80%) selesai.")
print("Jumlah total data:", len(df))
print("Jumlah data Train (80%):", len(train_df))

# Tampilkan 5 baris teratas dari Train
display(train_df.head())
```

Step 1: Buat Train (80%) selesai.
Jumlah total data: 731
Jumlah data Train (80%): 584

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
682	683	2012-11-13	4	1	11	0	2	1	2	0.343333	0.323225	0.662917	0.342046	327	3767	4094
250	251	2011-09-08	3	0	9	0	4	1	3	0.633913	0.555361	0.939565	0.192748	153	1689	1842
336	337	2011-12-03	4	0	12	0	6	0	1	0.299167	0.310604	0.612917	0.095783	706	2908	3614
260	261	2011-09-18	3	0	9	0	0	0	1	0.507500	0.490537	0.695000	0.178483	1353	2921	4274
543	544	2012-06-27	3	1	6	0	3	1	1	0.697500	0.640792	0.360000	0.271775	1077	6258	7335

- Fungsi `train_test_split`: Fungsi ini berasal dari *library* Scikit-learn (`sklearn`) dan digunakan untuk membagi dataset secara acak. Proporsi Pembagian: Dataset dibagi dengan proporsi 80% untuk data latih dan 20% untuk data uji. Ini diatur melalui parameter `test_size=0.2`.
- Parameter Penting:
 - `shuffle=True`: Memastikan data diacak sebelum dibagi untuk menghindari bias jika data terurut berdasarkan pola tertentu.
 - `random_state=42`: Berfungsi sebagai "kunci" untuk pengacakan. Dengan menetapkan nilai ini, hasil pembagian data akan selalu sama setiap kali kode dijalankan, sehingga eksperimen menjadi reproduibel (dapat diulang dengan hasil yang sama).
- Hasil: Dari total 731 data, sebanyak 584 data dialokasikan sebagai `train_df` (data latih). Sisa 147 data akan menjadi `test_df` (data uji).

3. Menampilkan Data Valid dari 10% Data Train

```
# Ambil 10% dari train_df sebagai validation
# (train_df di-reassign menjadi train tanpa bagian validation)
train_df, val_df = train_test_split(train_df, test_size=0.1, random_state=RANDOM_STATE, shuffle=True)

print("Step 2: Buat Validation (10% dari Train) selesai.")
print("Jumlah data Train (setelah diambil validation):", len(train_df))
print("Jumlah data Validation (10% dari Train awal):", len(val_df))

# Tampilkan 5 baris teratas dari Validation
display(val_df.head())
```

Step 2: Buat Validation (10% dari Train) selesai.
Jumlah data Train (setelah diambil validation): 525
Jumlah data Validation (10% dari Train awal): 59

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
325	326	2011-11-22	4	0	11	0	2	1	3	0.416667	0.421696	0.962500	0.118792	69	1538	1607
410	411	2012-02-15	1	1	2	0	3	1	1	0.348333	0.351629	0.531250	0.181600	141	4028	4169
92	93	2011-04-03	2	0	4	0	0	0	1	0.378333	0.378767	0.480000	0.182213	1651	1598	3249
47	48	2011-02-17	1	0	2	0	4	1	1	0.435833	0.428658	0.505000	0.230104	259	2216	2475
508	509	2012-05-23	2	1	5	0	3	1	2	0.621667	0.584612	0.774583	0.102000	766	4494	5260

- **Sumber Data:** Data validasi diambil dari `train_df` (data latih awal yang berjumlah 584). Proporsi: Sebanyak 10% dari `train_df` diambil untuk menjadi `val_df` (data

validasi). Ini diatur dengan

`test_size=0.1`.

- **Hasil:**

- Data validasi (`val_df`) kini berjumlah 59 data.
- Data latih (`train_df`) yang tersisa (setelah diambil untuk validasi) menjadi 525 data.

4. Menampilkan Data Train (20% dari Total Data)

```
# Tampilkan Test yang sudah disisihkan di Step 1
print("Step 3: Testing set (20% dari total)")
print("Jumlah data Testing:", len(test_df))
display(test_df.head())

# Final sanity check: jumlah seluruh subset harus sama dengan total dataset
total = len(train_df) + len(val_df) + len(test_df)
print("\nTotal (train + val + test) =", total)
print("Total dataset asli      =", len(df))
assert total == len(df), "ERROR: Jumlah train+val+test tidak sama dengan total dataset!"
print("✅ Pembagian berhasil dan jumlahnya cocok dengan total dataset.")
```

Step 3: Testing set (20% dari total)
Jumlah data Testing: 147

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	temp	atemp	hum	windspeed	casual	registered	cnt
703	704	2012-12-04	4	1	12	0	2	1	1	0.475833	0.469054	0.733750	0.174129	551	6055	6606
33	34	2011-02-03	1	0	2	0	4	1	1	0.186957	0.177878	0.437826	0.277752	61	1489	1550
300	301	2011-10-28	4	0	10	0	5	1	2	0.330833	0.318812	0.585833	0.229479	456	3291	3747
456	457	2012-04-01	2	1	4	0	0	0	2	0.425833	0.417287	0.676250	0.172267	2347	3694	6041
633	634	2012-09-25	4	1	9	0	2	1	1	0.550000	0.544179	0.570000	0.236321	845	6693	7538

Total (train + val + test) = 731
Total dataset asli = 731
✅ Pembagian berhasil dan jumlahnya cocok dengan total dataset.

- **Inspeksi Data Uji:** Data uji (`test_df`) yang merupakan 20% dari total dataset ditampilkan. Jumlah datanya adalah 147. Data ini akan digunakan hanya pada tahap evaluasi final model.
- **Sanity Check:** Dilakukan verifikasi dengan menjumlahkan semua subset data (`train_df`, `val_df`, dan `test_df`). Totalnya harus sama dengan jumlah data pada dataset asli.
- **Hasil Pengecekan:** Total data dari gabungan ketiga subset adalah 731, yang cocok dengan jumlah data asli. Perintah `assert` memastikan bahwa jika jumlahnya tidak cocok, program akan berhenti dan menampilkan error. Dalam kasus ini, proses pembagian dinyatakan berhasil.

5. Membuat rangkuman pembagian dataset

Tahap ini memberikan gambaran yang jelas mengenai hasil akhir dari seluruh proses pemisahan data

```
print("📊 Rangkuman Pembagian Dataset:")
print("Training set   :", len(train_df))
print("Validation set  :", len(val_df))
print("Testing set     :", len(test_df))
print("Total           :", len(train_df) + len(val_df) + len(test_df))
```

📊 Rangkuman Pembagian Dataset:
Training set : 525
Validation set : 59
Testing set : 147
Total : 731